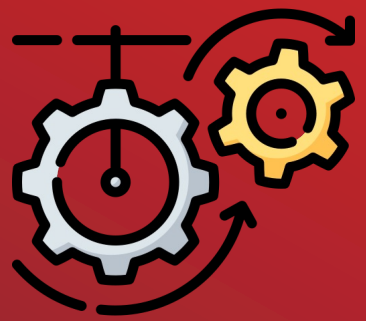
JPA



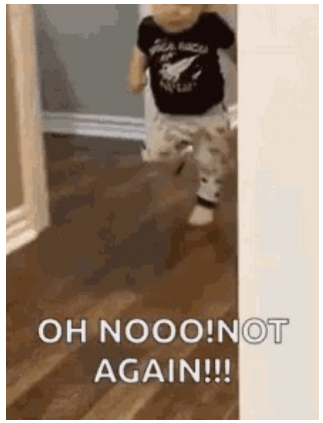
zenika MIAGE RÉSEAU DES MIAGE DE FRANCE Grenoble UGA UFR IM²AG Université Grenoble Alpes

Programme

BDSI

3 CM
- JPA*2
- Spring

5 TP
- JPA*3
- JPA/Spring
- Spring



VA

2 CM
- Spring / Test serveur
- Test serveur

4 TP
- Spring/Test
- Test serveur

Différence entre DAO et JPA

zenika
MIAGE Grenoble RÉSEAU DES MIAGE DE FRANCE
UGA UFR IM²AG Université Grenoble Alpes

DAO



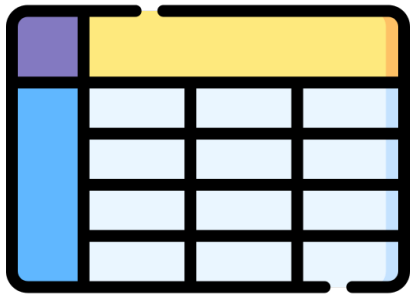
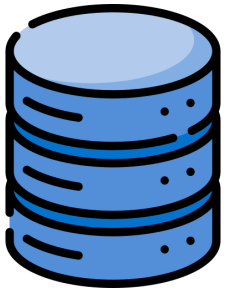


Table en base



Fichier java



Base de données

- Représenter les données de la table
- Créer les actions de base
- CRUD (CREATE, READ, UPDATE, DELETE)
- Gérer les relations entre tables
- Gérer les cascades

⚠ et dépendant du SGBD !

JPA

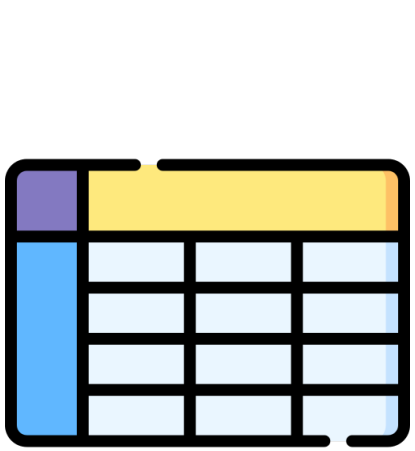
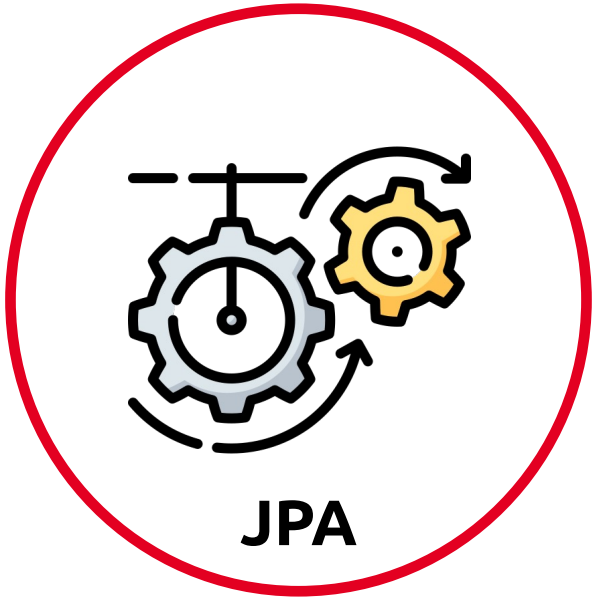


Table en base



Fichier java



JPA



Base de données

- Déclaration du modèle
- Déclaration des relations

- Fait le CRUD de lui-même
- Gère la connexion entre les objets java et la base
- Etc…

Object Relationship Mapping (ORM)



zenika

MIAGE Grenoble RÉSEAU DES MIAGE DE FRANCE

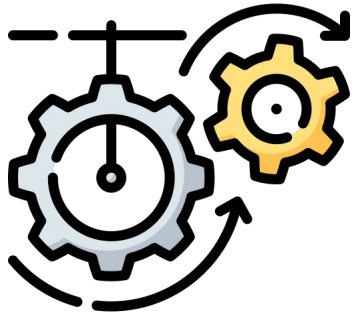
UGA UFR IM²AG Université Grenoble Alpes

Object Relationship Mapping (ORM)



Modèle applicatif

Transforme



ORM

HIBERNATE

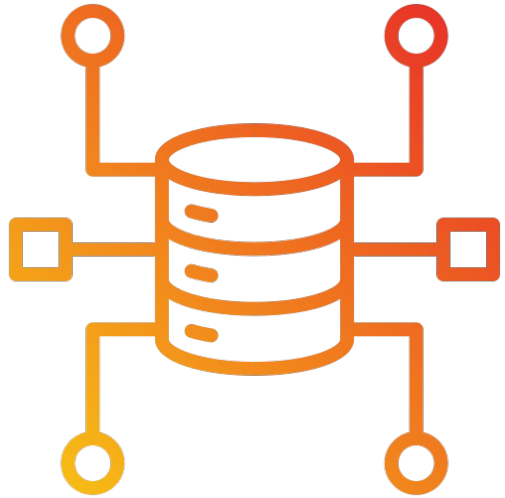
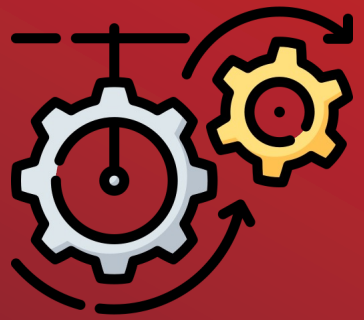


Schéma relationnel

JPA : Java Persistence API



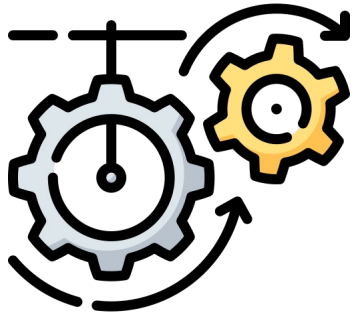
zenika MIAGE Grenoble RÉSEAU DES MIAGE DE FRANCE UGA UFR IM²AG Université Grenoble Alpes

Object Relationship Mapping (ORM)



Modèle applicatif

Transforme



ORM

HIBERNATE

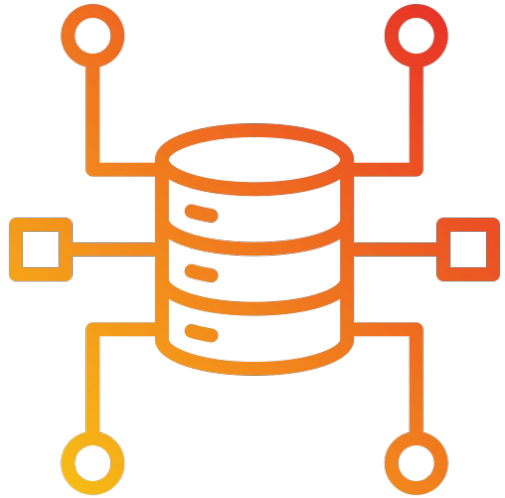


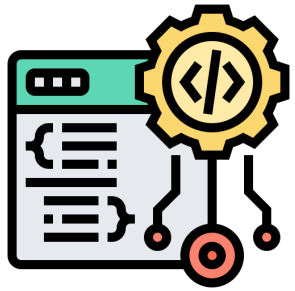
Schéma relationnel

Java Persistante API

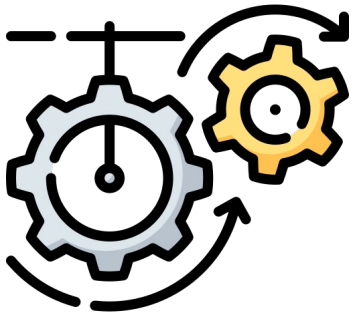


Modèle applicatif

Modélisé



JPA



ORM

HIBERNATE

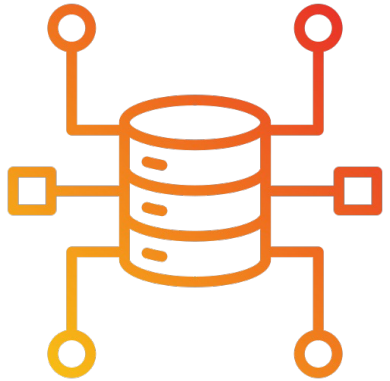
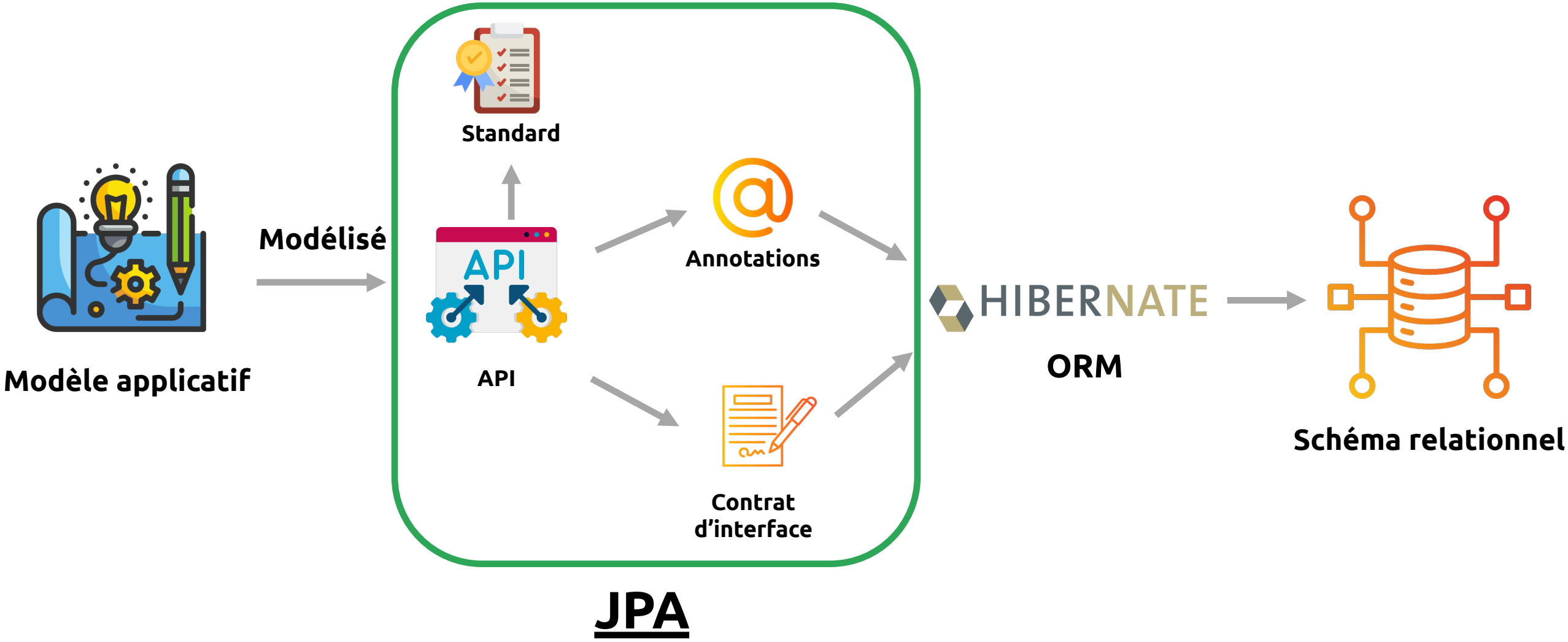
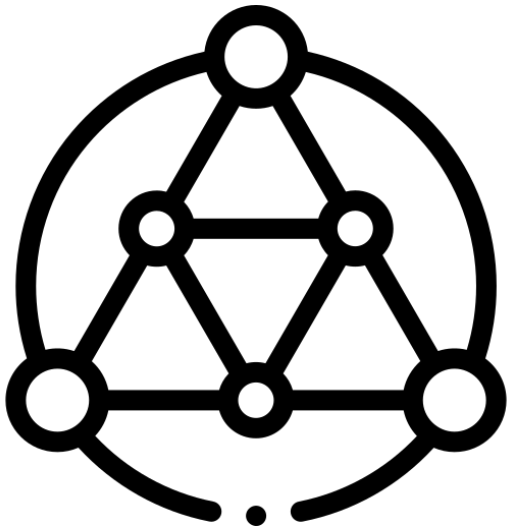


Schéma relationnel

Java Persistante API



Ses avantages



Réduit la complexité du code

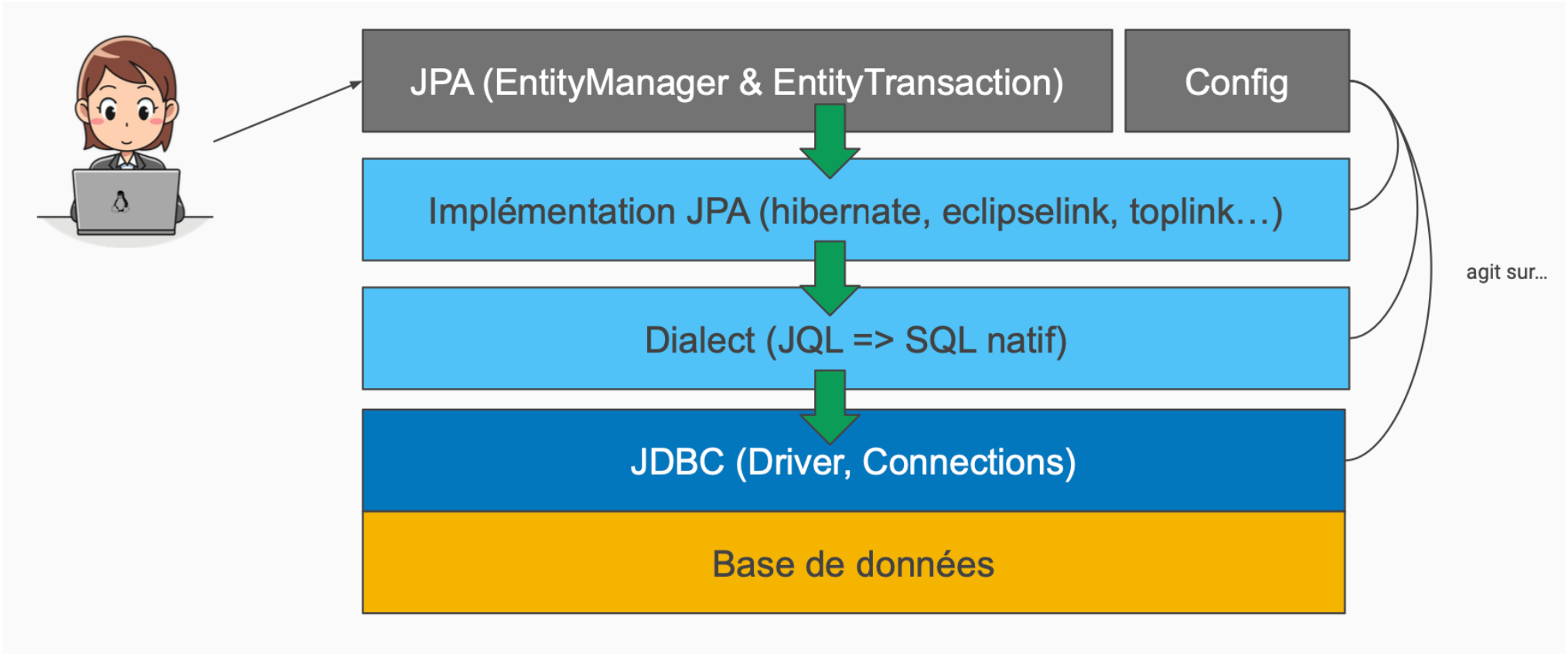


Réduit la lourdeur du code

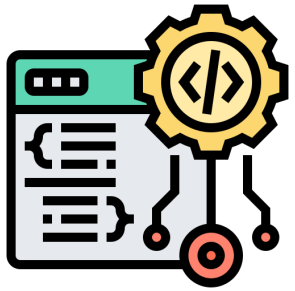


Nous permet de ne plus écrire du SQL directement

Architecture

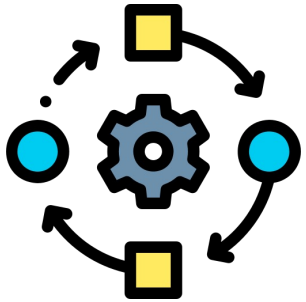


Remarque

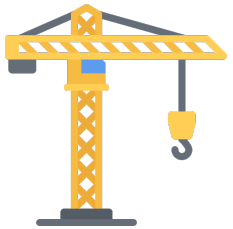


JPA

2 Paradigmes



S'adapte à un schéma existant (*validate*)

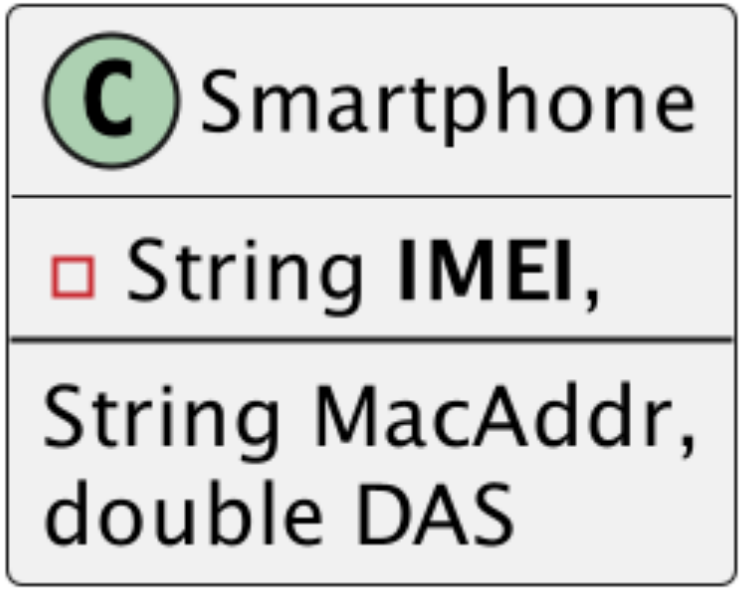


Crée un schéma en fonction du mapping (*create*)

Mapping d’une entité simple

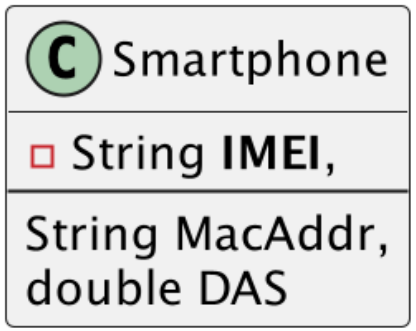
zenika
MIAGE Grenoble RÉSEAU DES MIAGE DE FRANCE
UGA UFR IM²AG Université Grenoble Alpes

Création d'une entité



@Entity

class




```
@Entity(name = "iphone_for_ever")
public class SmartphoneEntity{
}
```



Rendre persistant

Attribut

Name

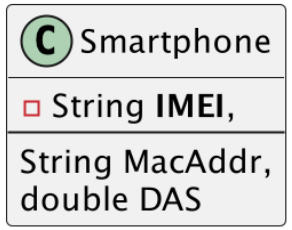
- Donner un nom à l'entité

Remarque :

- En mode **create** , si le **name** n'est pas renseigné alors le nom de la table en base sera le nom de la classe en snake case.
  Dans cet exemple => « smartphone_entity »
- Sinon si l'attribut **name** est déclaré ce sera lui qui sera utilisé pour le nom de la table en base, dans cet exemple => « iphone_for_ever »

@Table

class

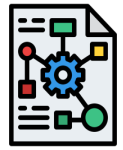



```
@Entity
@Table(name = "nomTableExistante")
public class SmartphoneEntity{
}
```

Attribut



Les index



Le schéma

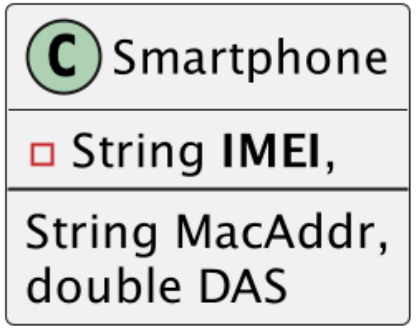


Les clés uniques

Name	Schema	Indexes	UniqueConstraints
○ Déclaration du nom de la table ○ Important si la table existe en base	○ Déclaration du schéma en base	○ Déclaration des indexes de la table	○ Déclaration des clés uniques

@Id

attribute




```java
@Entity
@Table(name ="nomTableExistante")
public class SmartphoneEntity{
    @Id
    private String imei;
}
```

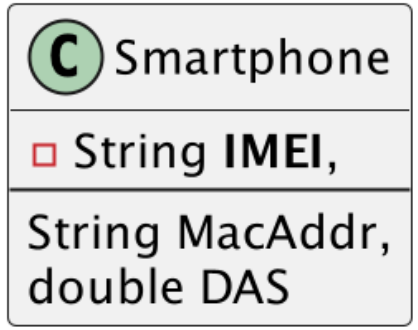


Clé Primaire

⚠️ **Cette annotation est obligatoire pour JPA**

@Column

attribute




```java
@Entity(name = "iphoneForEver")
public class SmartphoneEntity{
    @Id
    @Column (name="imei")
    private String imei;
}
```



Déclarer une colonne et ses particularités

Remarques :

- Plus ou moins obligatoire en **validate**
  - Documentation
  - Lisibilité
  - Bonne association
- Inutile en mode **create**, sauf si :
  - Déclarer une particularité à votre classe

Implémentation finale

En mode *validate*

En BD



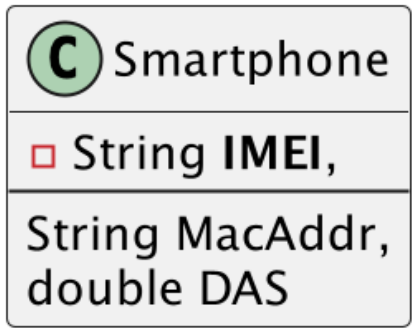

```java
@Table(name = "iphone_for_ever")
@Entity
public class SmartphoneEntity{
   @Id
   @Column(name = "imei")
   private String imei;

   @Column(name = "mac_ip")
   private String macAddr;

   @Column(name =  »mobile_das",updatable = false,nullable = false)
   private double das;
}
```

Implémentation finale

En mode *create*

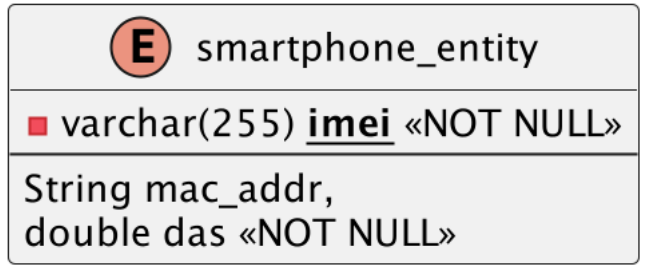



```
@Entity
public class SmartphoneEntity{
    @Id
    private String imei;

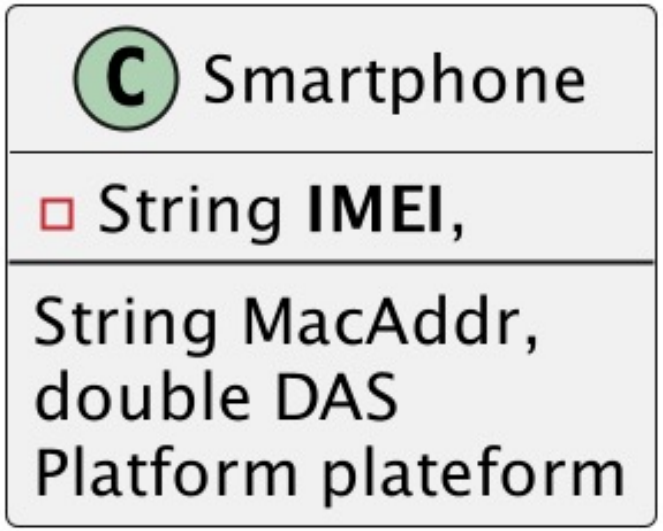
    private String macAddr;

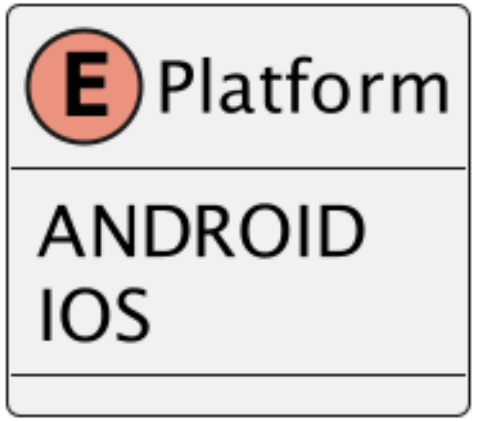
    @Column(updatable = false, nullable = false)
    private double das;
}
```

En BD



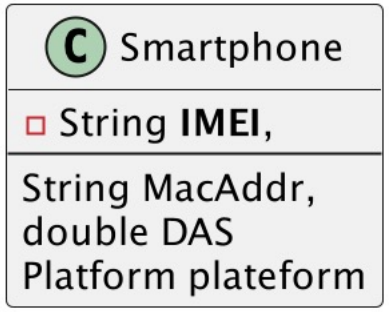
Les types enums



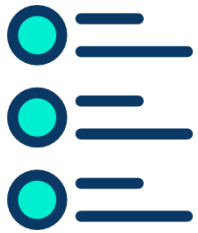


@Enumerated

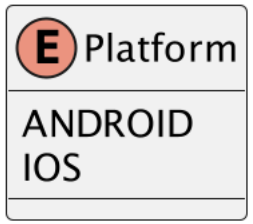
attribute




```
@Entity
public class SmartphoneEntity{
   //Before
    @Enumerated(EnumType.ORDINAL)
    Platform platform;
}
```



Donne la gestion d'un enum java en base



2 Types possibles

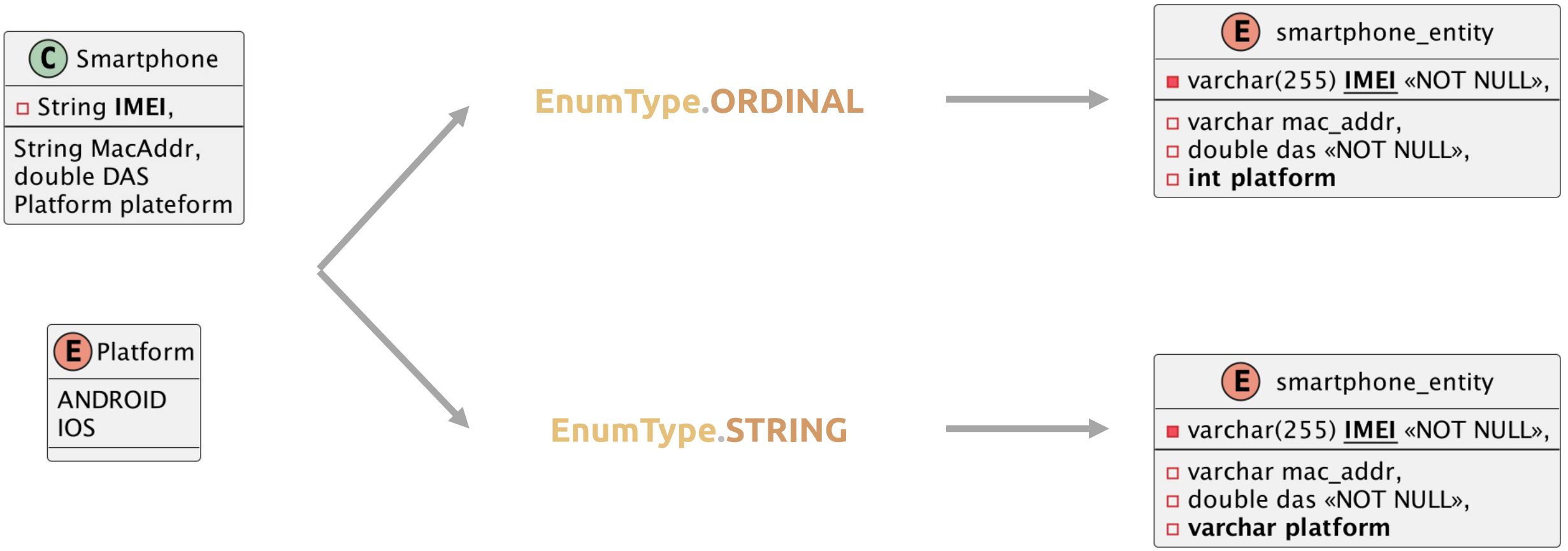
EnumType.ORDINAL

- Utilise l'ordinal de l'enum (int) pour le représenter en base

EnumType.STRING

- Utilise le nom de l'enum (varchar) pour le représenter en base

Implémentation



Règle d'or

POJO ONLY

(Plain Old Java Object)



- getter
- setter
- hashCode()
- equals()



- adder ()
- Synchronisation
- toString()



- Pas de méthodes intelligentes

Héritage

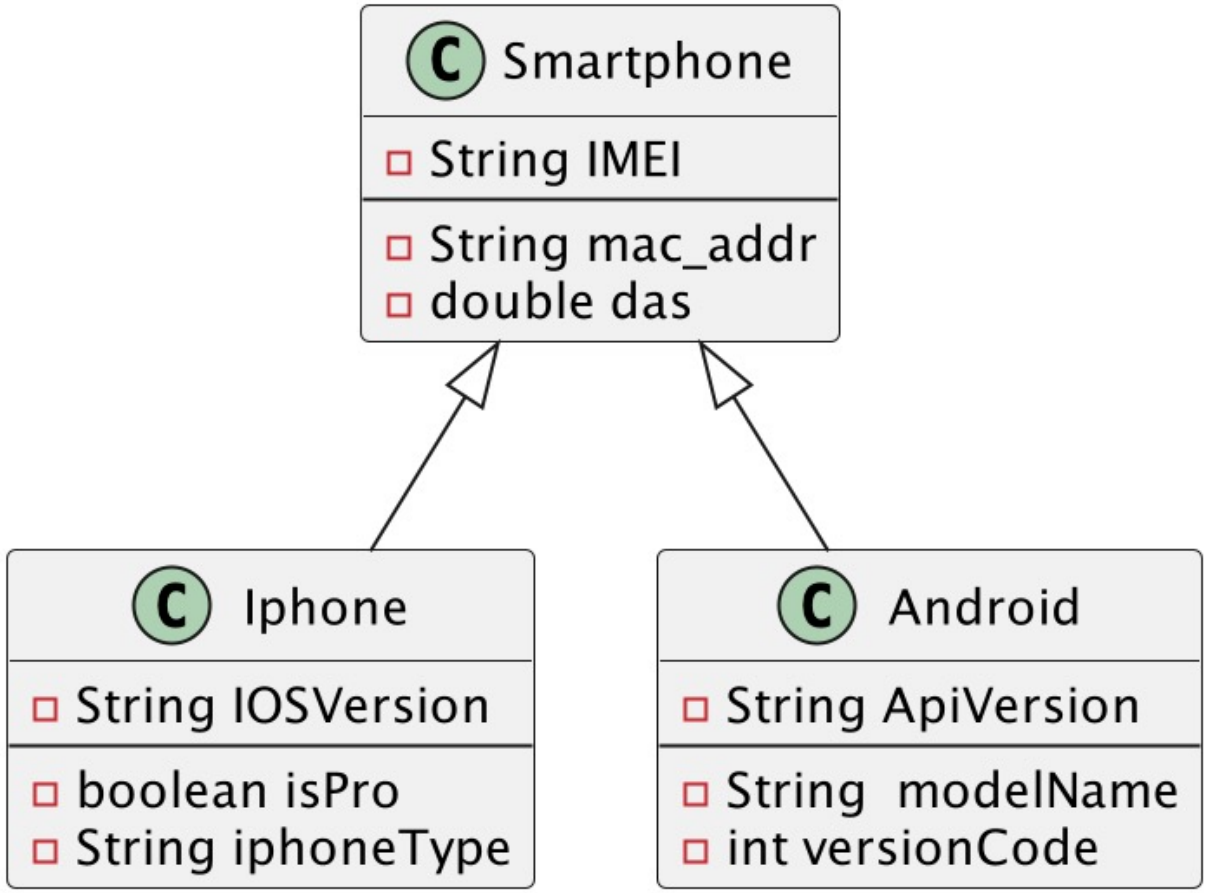


zenika

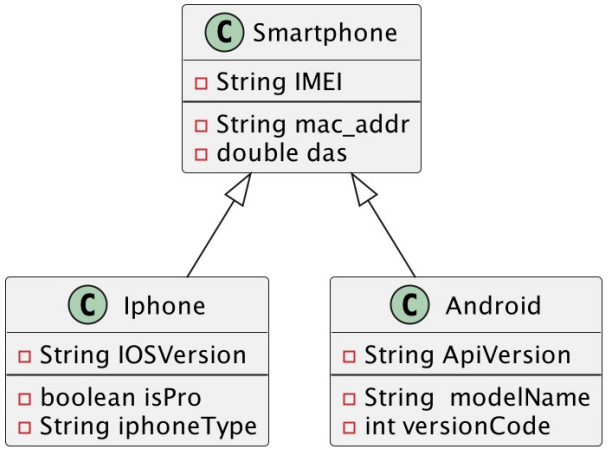
MIAGE Grenoble RÉSEAU DES MIAGE DE FRANCE

UGA UFR IM²AG Université Grenoble Alpes

Héritage


Smartphone
String IMEI
String mac_addr
double das
Iphone
String IOSVersion
boolean isPro
String iphoneType
Android
String ApiVersion
String modelName
int versionCode

@Inheritance
classe mère




```java
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
public abstract class Smartphone {
    @Id
    private String imei;
    private String mac_addr;
    private double das;
}
```



Déclaration d'héritage

Stategy

- Permet de donner la stratégie de gestion de l'héritage de cette classe à JPA

3 stratégies possibles

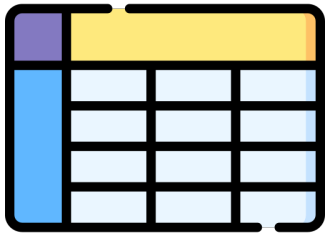


Table unique

Utiliser un champ de la table pour différencier la classe fille

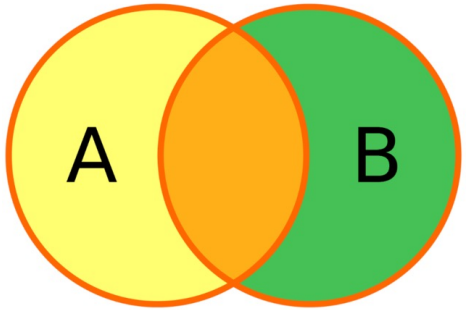
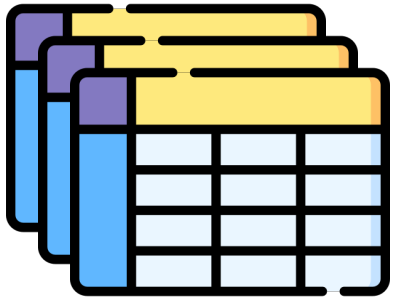


Table de jointure

Créer une table jointure pour différencier la classe fille



Une table par classe

Créer une table par classe fille

Implémentation

SINGLE_TABLE

Classe mère

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name="type")
public abstract class Smartphone {
   @Id
   private String imei;
   private String mac_addr;
   private double das;
}
```

→ `@DiscriminatorColumn(name="type")`

↓

Créer la colonne qui va permettre la différentiation

Classe fille

```
@Entity
@DiscriminatorValue("I")
public class Iphone extends Smartphone{
   String IosVersion;
   boolean isPro;
   String iphoneType;
}
```

`@DiscriminatorValue("")`

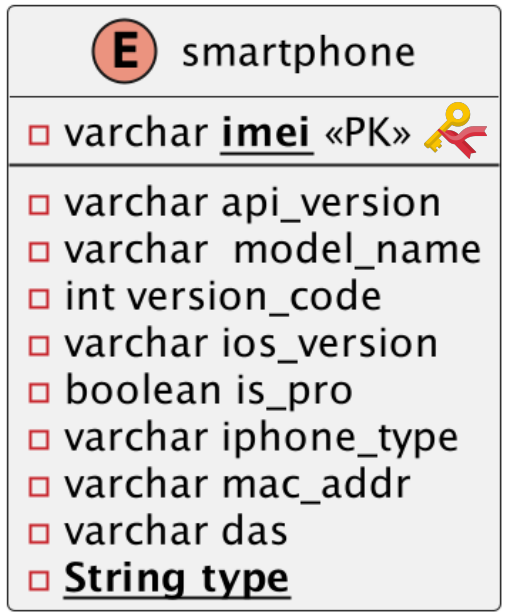
↓

Permet l'association entre la colonne type et ses valeurs

Classe fille

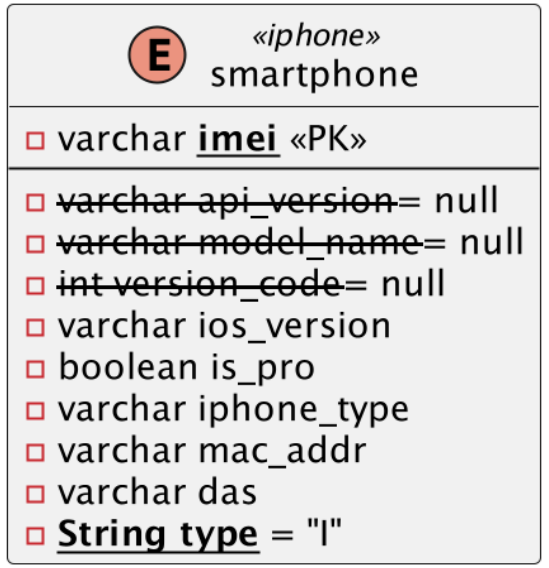
```
@Entity
@DiscriminatorValue("A")
public class Android extends Smartphone{
   String ApiVersion;
   String modelName;
   int versionCode;
}
```

Représentation en base

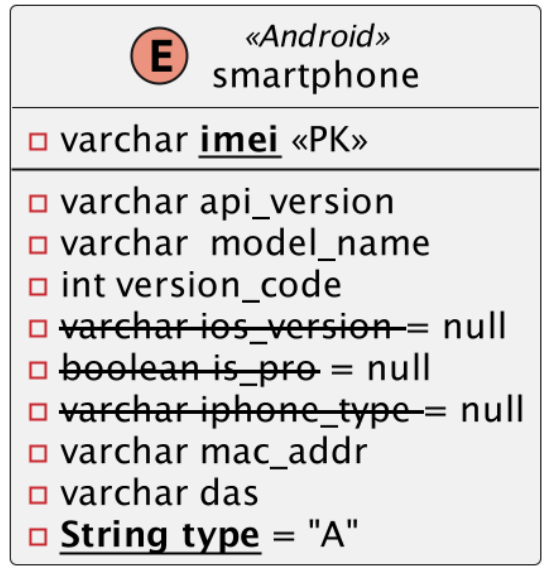


Un Iphone

Un Android



- La valeur « type » vaut **I**, donc la classe fille associée est **Iphone**
- Les champs correspondant à un Android seront forcément ***null***



- La valeur « type » vaut **A**, donc la classe fille associée est **Android**
- Les champs correspondant à un iphone seront forcément ***null***

Implémentation

JOINED

Classe mère

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
@DiscriminatorColumn(name="type")
public abstract class Smartphone {
    @Id
    private String imei;
    private String mac_addr;
    private double das;
}
```

Classe fille

```
@Entity
@DiscriminatorValue("I")
public class Iphone extends Smartphone{
    String IosVersion;
    boolean isPro;
    String iphoneType;
}
```

```
@DiscriminatorValue("")
```

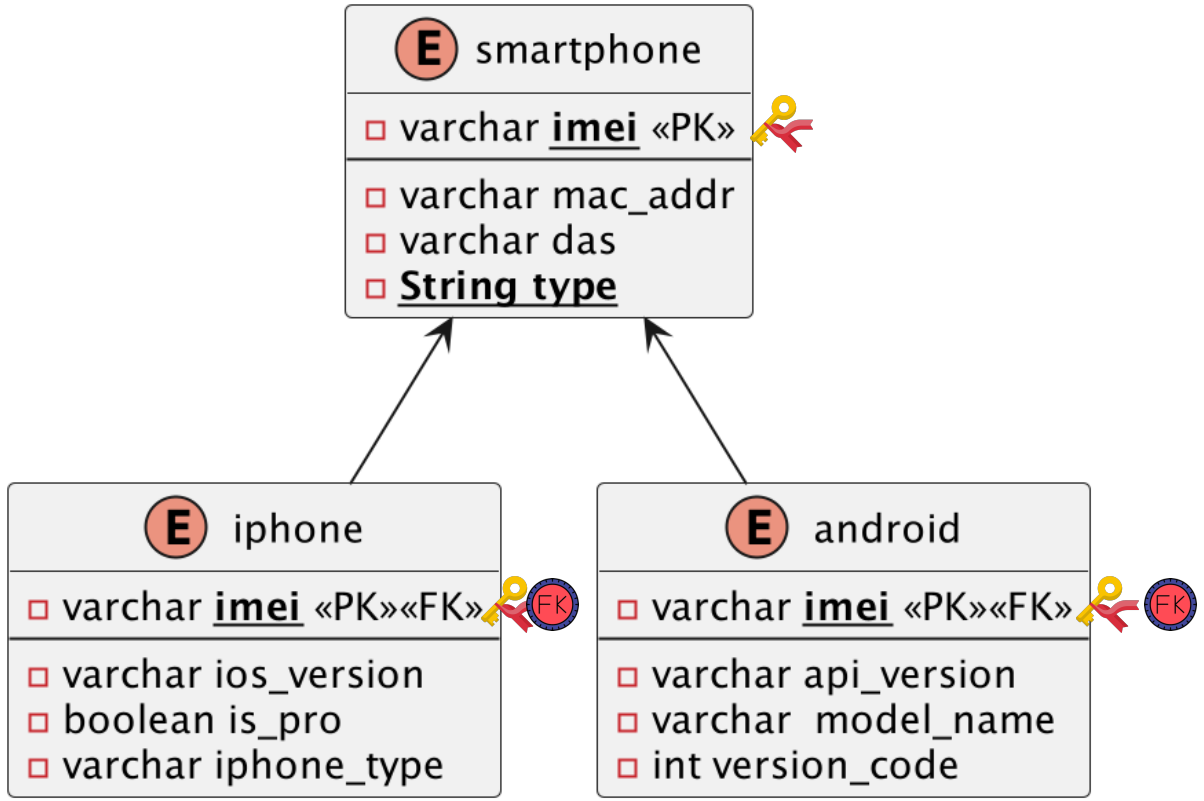


Permet l'association entre la colonne type et ses valeurs

Classe fille

```
@Entity
@DiscriminatorValue("A")
public class Android extends Smartphone{
    String ApiVersion;
    String modelName;
    int versionCode;
}
```

Représentation en base



Un Iphone

Un Android

- *Si **type** vaut **I**, alors une requête avec une jointure sur la table **iphone** va être faite pour récupérer toutes les informations*

- *Si **type** vaut **A**, alors une requête avec une jointure sur la table **Android** va être faite pour récupérer toutes les informations*

Implémentation

TABLE_PER_CLASS
Standard

Classe mère

```
@MappedSuperclass
public abstract class Smartphone {
    @Id
    private String imei;
    private String mac_addr;
    private double das;
}
```

Classe fille

```
@Entity
public class Iphone extends Smartphone{
    String IosVersion;
    boolean isPro;
    String iphoneType;
}
```

@MappedSuperclass



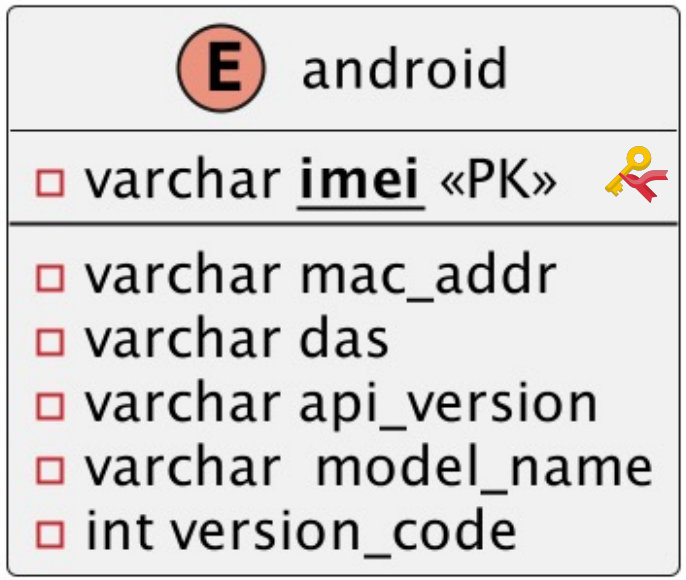
Permet de donner l'indication que la classe mère ne peut jamais être concrète

Classe fille

```
@Entity
public class Android extends Smartphone{
    String ApiVersion;
    String modelName;
    int versionCode;
}
```

Représentation en base







Ici une table par classe est créée, et aucune pour la classe mère.

Cependant !



zenika MIAGE Réseau des MIAGE de France Grenoble UGA UFR IM²AG Université Grenoble Alpes

Implémentation
TABLE_PER_CLASS
Polymorphique

Classe mère

```
@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public class Smartphone {
    @Id
    private String imei;
    private String mac_addr;
    private double das;
}
```

Classe fille

```
@Entity
public class Iphone extends Smartphone{
    String IosVersion;
    boolean isPro;
    String iphoneType;
}
```

```
@Inheritance(strategy =
InheritanceType.TABLE_PER_CLASS)
```

Permet de donner l'indication d'une table par classe et de voir la classe mère comme une entité à part
Cela permet les requêtes polymorphiques

Classe fille

```
@Entity
public class Android extends Smartphone{
    String ApiVersion;
    String modelName;
    int versionCode;
}
```

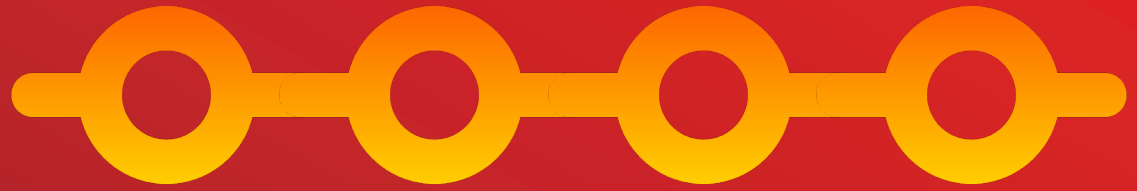
A votre tour !



zenika

MIAGE Grenoble RÉSEAU DES MIAGE DE FRANCE

UGA UFR IM²AG Université Grenoble Alpes



Les états des entités dans l'entity Manager

zenika

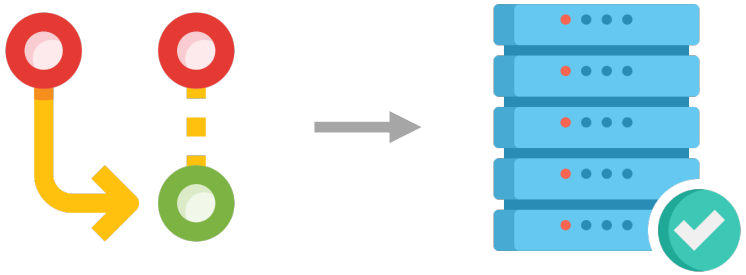
MIAGE Grenoble RÉSEAU DES MIAGE DE FRANCE

UGA UFR IM²AG Université Grenoble Alpes

PERSIST

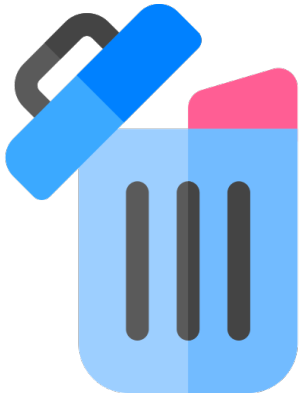


L’objet devient persistant

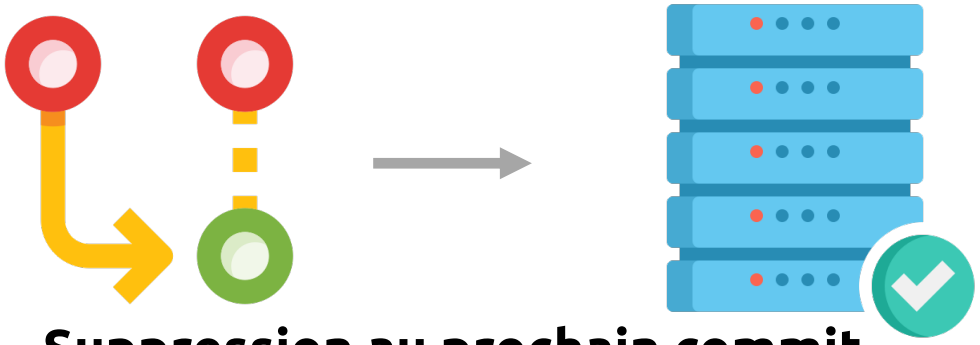


Ajout à la base au prochain commit (insert, update)

REMOVE



Suppression de l'objet

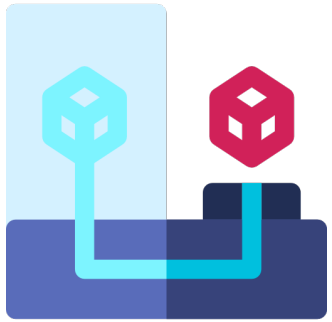


Suppression au prochain commit (delete)

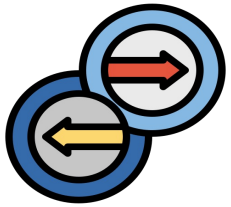
DETACH



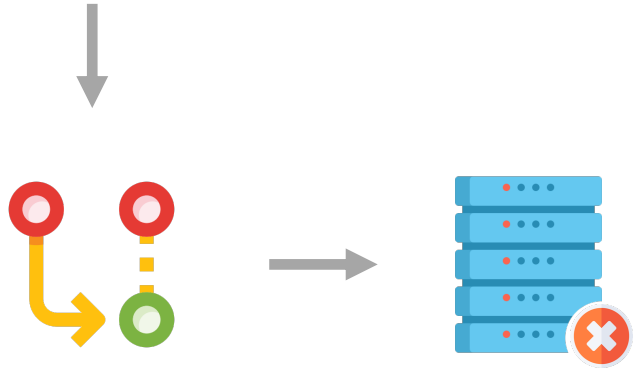
N’est plus synchronisé



Si vous recherchez l’objet dans le l’entity manager, vous aurez un clone de cet objet



Modification



Les modifications ne sont pas prises en compte

REFRESH







Synchronise l'objet avec la table

Dans le cas d'une modification concurrente de la même entité

MERGE



1

Changement d'entity manager

⚠ **Peu fréquent**



2

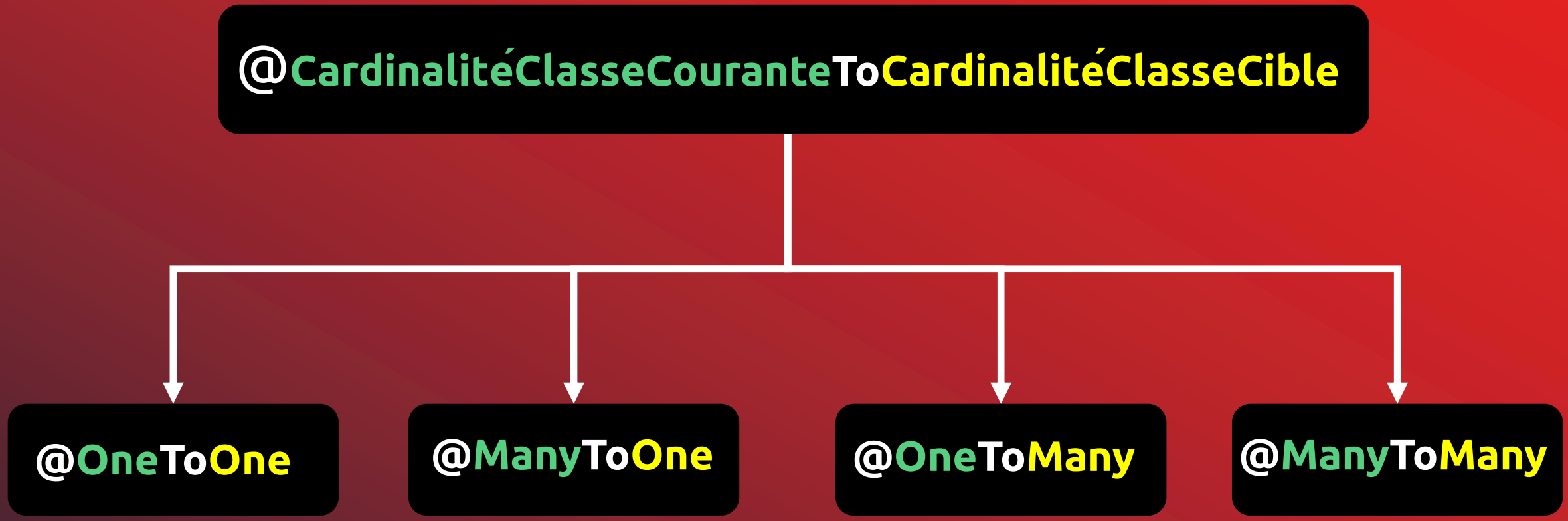
Encore vivant ?



zenika MIAGE Réseau des MIAGE de France Grenoble UGA UFR IM²AG Université Grenoble Alpes

@Relation

@Relation


@CardinalitéClasseCourantetoCardinalitéClasseCible
@OneToOne
@ManyToOne
@OneToMany
@ManyToMany

De 2 types





Unidirectionnelle

Bidirectionnelle

zenika

MIAGE Grenoble RÉSEAU DES MIAGE DE FRANCE

UGA UFR IM²AG Université Grenoble Alpes

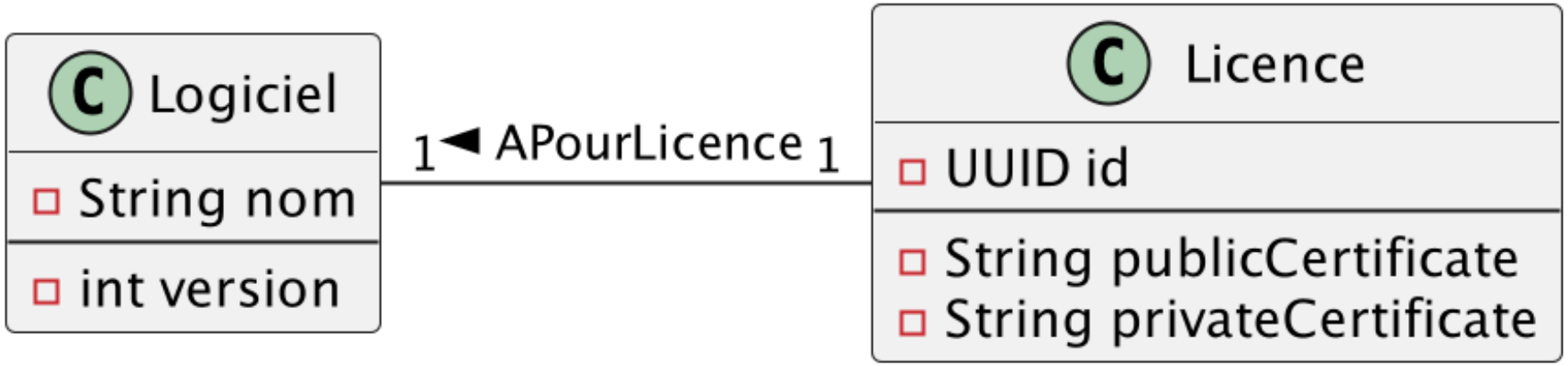
@OneToOne

zenika

MIAGE Grenoble RÉSEAU DES MIAGE DE FRANCE

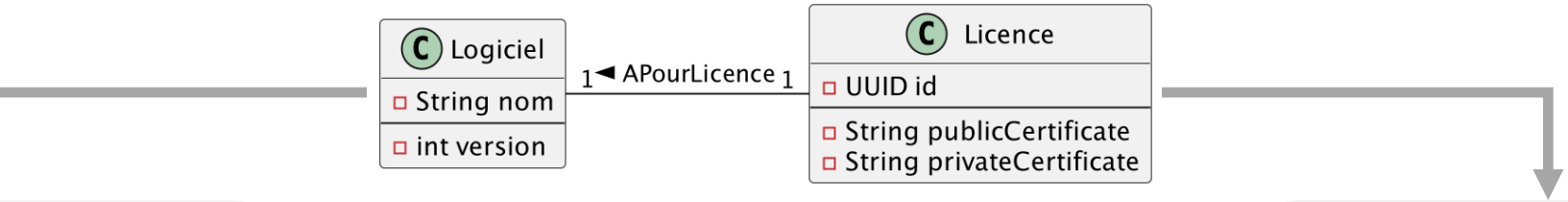
UGA UFR IM²AG Université Grenoble Alpes

@OneToOne


Logiciel
String nom
int version
1
APourLicence
1
Licence
UUID id
String publicCertificate
String privateCertificate

@OneToOne

Unidirectionnelle



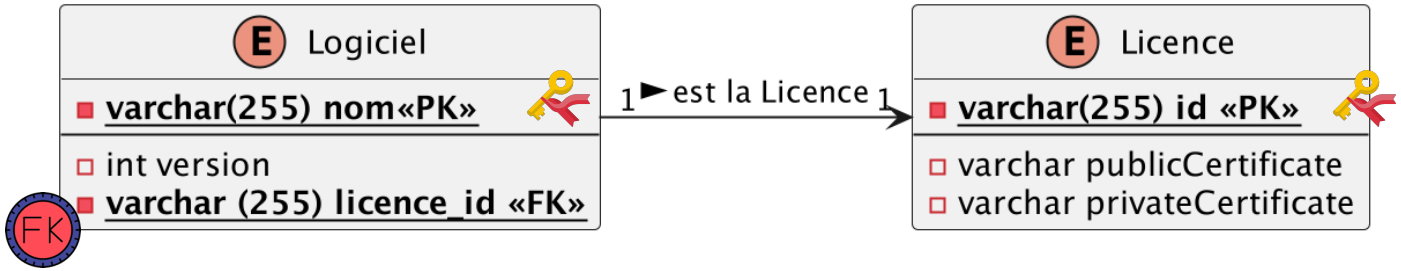

```java
@Entity
public class LogicielEntity{
    @Id
    private String name ;

    private int version;

    @OneToOne
    private LicenceEntity licence;
}
```

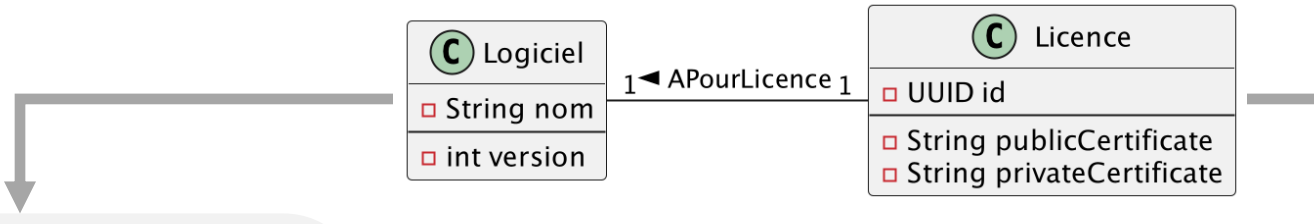
```java
@Entity
public class LicenceEntity{
    @Id
    private UUID uuid;
    private String publicCertificate;
    private String privateCertificate;

}
```



@OneToOne

Bidirectionnelle



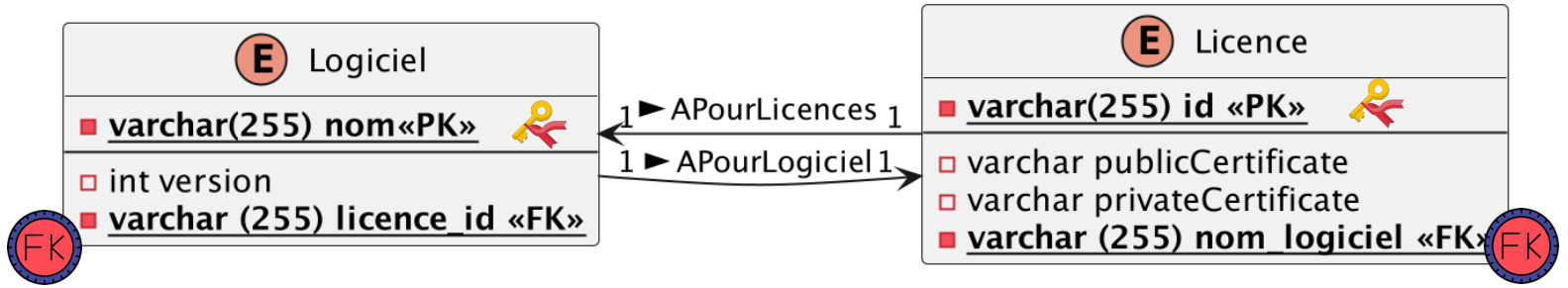

```java
@Entity
public class LogicielEntity{
   @Id
   private String name ;

   private int version;

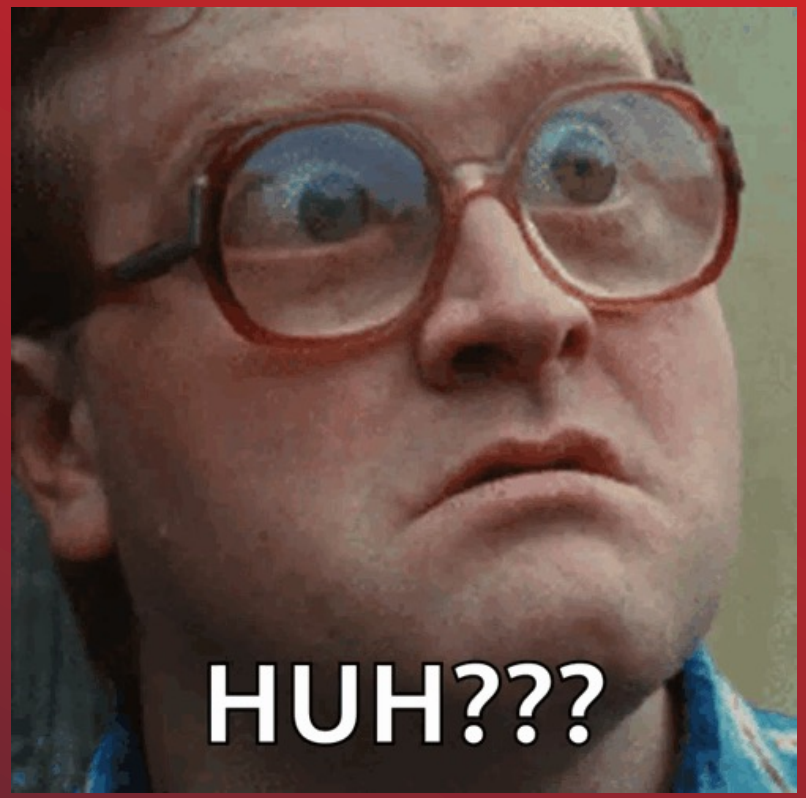
   @OneToOne
   private LicenceEntity licence;
}
```

```java
@Entity
public class LicenceEntity{
   @Id
   private UUID uuid;
   private String publicCertificate;
   private String privateCertificate;

   @OneToOne
   private LogicielEntity logiciel;
}
```



Vous n’avez pas vu quelque chose de bizarre ?



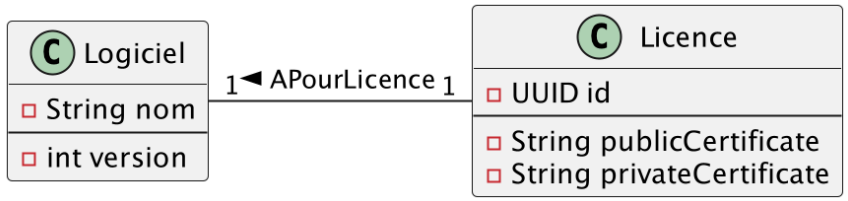
zenika

MIAGE RÉSEAU DES MIAGE DE FRANCE Grenoble

UGA UFR IM²AG Université Grenoble Alpes

@OneToOne

Bidirectionnelle




```java
@Entity
public class LogicielEntity{
    @Id
    private String name ;

    private int version;

    @OneToOne
    private LicenceEntity licence;
}
```

C'est Impossible



```java
@Entity
public class LicenceEntity{
    @Id
    private UUID uuid;
    private String publicCertificate;
    private String privateCertificate;

    @OneToOne
    private LogicielEntity logiciel;
}
```

Pas de panique, il y a une solution !



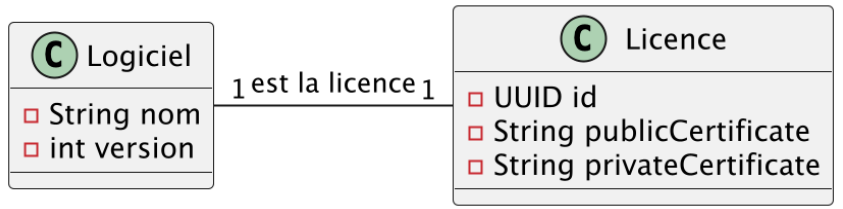
zenika

MIAGE Grenoble RÉSEAU DES MIAGE DE FRANCE

UGA UFR IM²AG Université Grenoble Alpes

@OneToOne

Bidirectionnelle

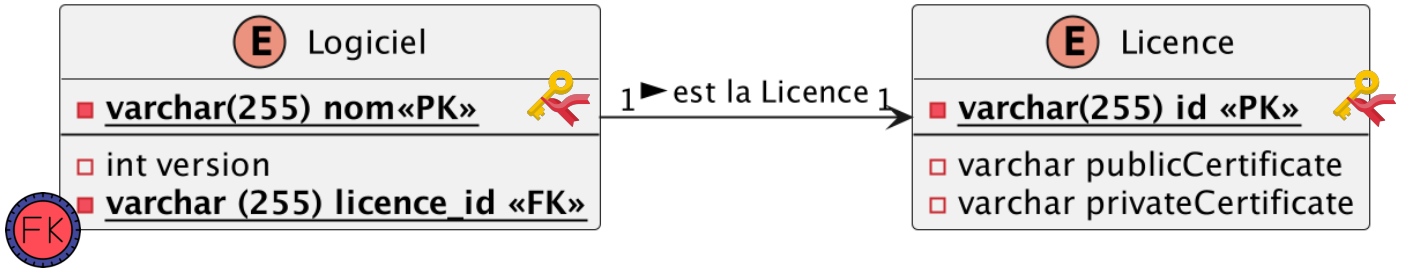



```java
@Entity
public class LogicielEntity{
   @Id
   private String name ;
   private int version;

   @OneToOne(mappedBy = "logiciel")
   private LicenceEntity licence;
}
```

```java
@Entity
public class LicenceEntity{
   @Id
   private UUID uuid;
   private String publicCertificate;
   private String privateCertificate;

   @OneToOne
   private LogicielEntity logiciel;
}
```



Votre cerveau va bien ?

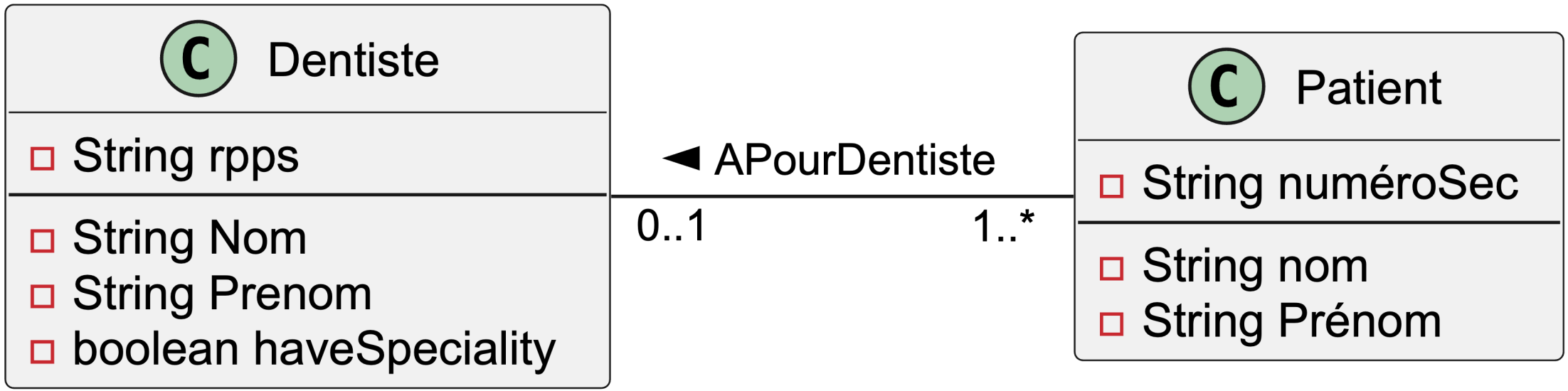


zenika MIAGE Réseau des MIAGE de France Grenoble UGA UFR IM²AG Université Grenoble Alpes

@ManyToOne

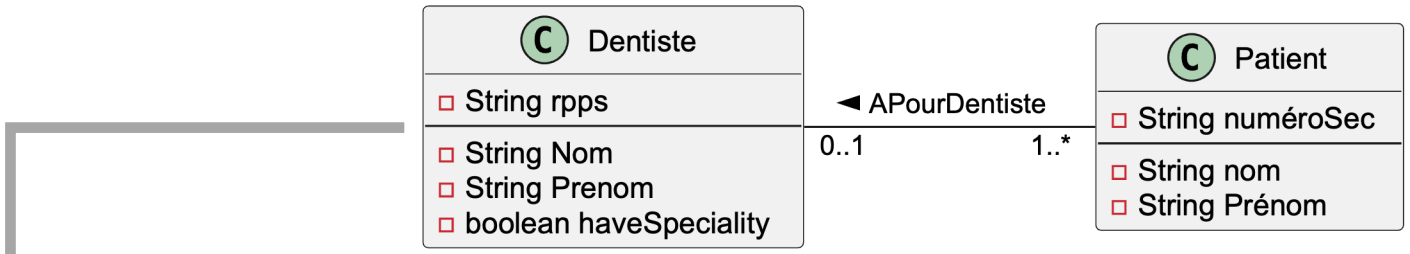
zenika
MIAGE Grenoble RÉSEAU DES MIAGE DE FRANCE
UGA UFR IM²AG Université Grenoble Alpes

@ManyToOne


Dentiste
String rpps
String Nom
String Prenom
boolean haveSpeciality
APourDentiste
0..1
1..*
Patient
String numéroSec
String nom
String Prénom

@ManyToOne

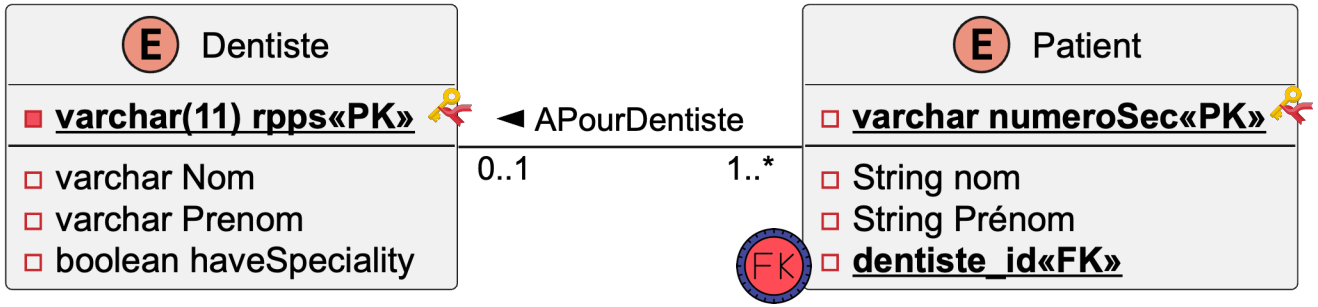
Unidirectionnelle




```java
@Entity
public class DentisteEntity{
    @Id
    private String rppd;
    private String nom;
    private String prenom;
    boolean haveSpeciality;
}
```

```java
@Entity
public class PatientEntity{
    @Id
    private String numeroSecu;
    private String nom;
    private String prenom;

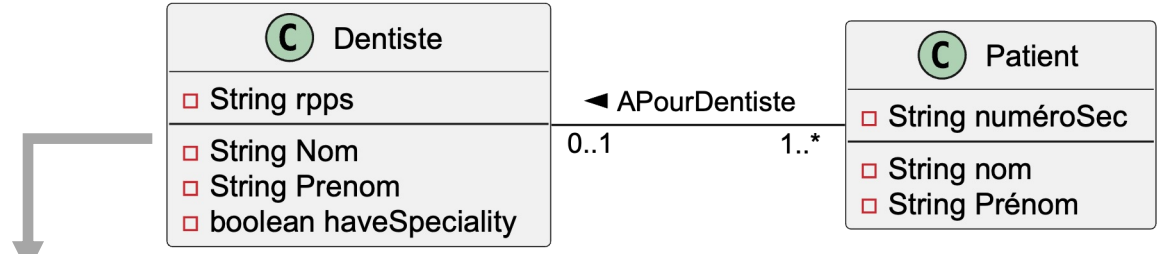
    @ManyToOne
    private DentisteEntity dentiste;
}
```



@OneToMany

@OneToMany

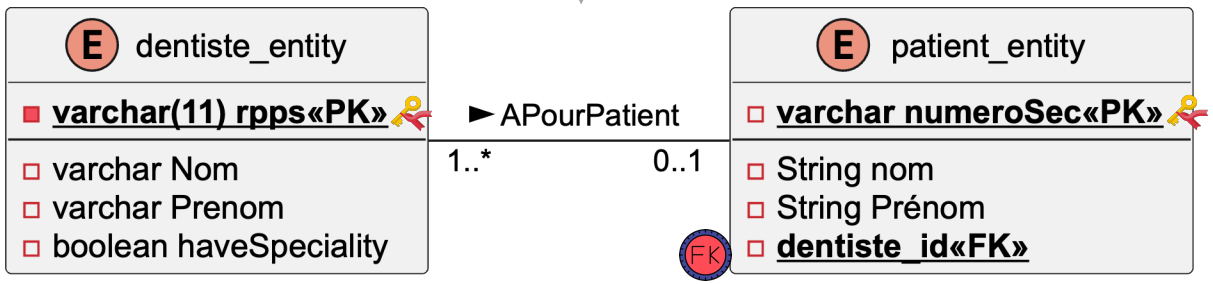
Unidirectionnelle




```java
@Entity
public class DentisteEntity{
   @Id
   private String rppd;
   private String nom;
   private String prenom;
   private boolean haveSpeciality;

@OneToMany
@JoinColumn(name = "dentiste_id", referencedColumnName =  "rppd")
private Set<PatientEntity> patients;
}
```

```java
@Entity
public class PatientEntity{
   @Id
   private String numeroSecu;
   private String nom;
   private String prenom;
}
```



Soufflez, vous êtes à la moitié



zenika

MIAGE Grenoble RÉSEAU DES MIAGE DE FRANCE

UGA UFR IM²AG Université Grenoble Alpes

Petit Quizz



zenika MIAGE Réseau des MIAGE de France Grenoble UGA UFR IM²AG Université Grenoble Alpes

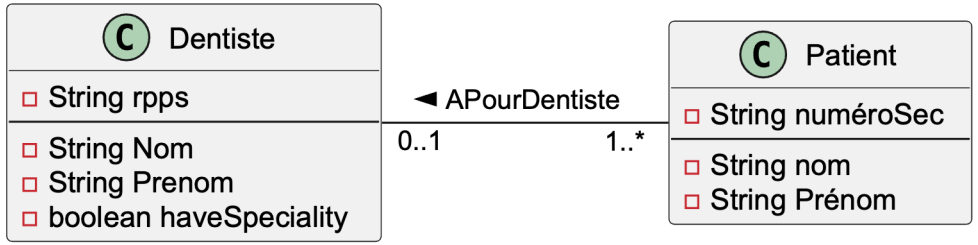
@ManyToOne
&
@OneToMany

Bidirectionnelle

zenika
MIAGE RÉSEAU DES MIAGE DE FRANCE Grenoble
UGA UFR IM²AG Université Grenoble Alpes

@ManyToOne / @OneToMany

Bidirectionnelle




```java
@Entity
public class DentisteEntity{
   @Id
   private String rppd;
   private String nom;
   private String prenom;
   boolean haveSpeciality;

   @OneToMany
   private Set<PatientEntity> patiens;
}
```



Bidirectionnelle

```java
@Entity
public class PatientEntity{
   @Id
   private String numeroSecu;
   private String nom;
   private String prenom;

   @ManyToOne
   private DentisteEntity dentiste;
}
```

Représentation en base

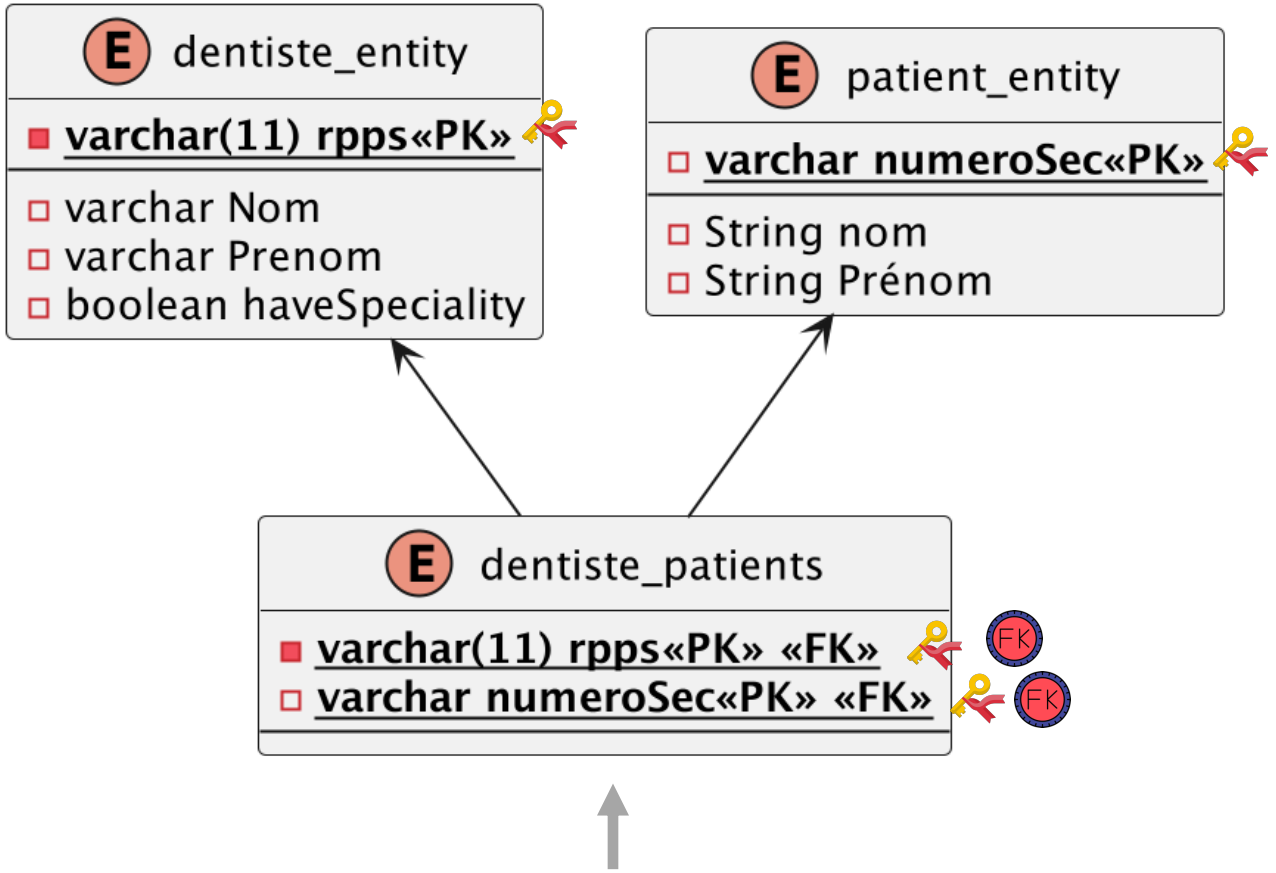
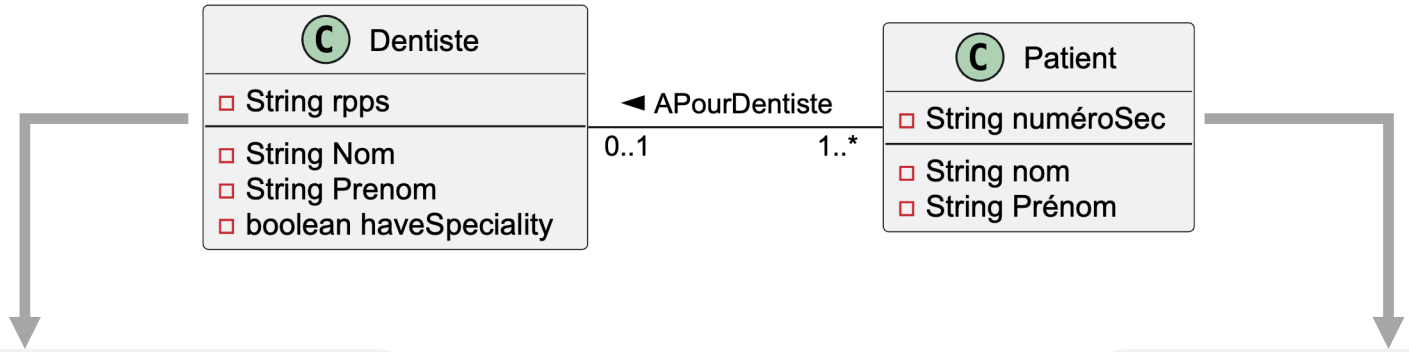


Table d'association

@ManyToOne / @OneToMany

Bidirectionnelle




```java
@Entity
public class DentisteEntity{
   @Id
   private String rppd;
   private String nom;
   private String prenom;
   boolean haveSpeciality;

   @OneToMany(mappedBy = "dentiste")
   private Set<PatientEntity> patiens;
}
```

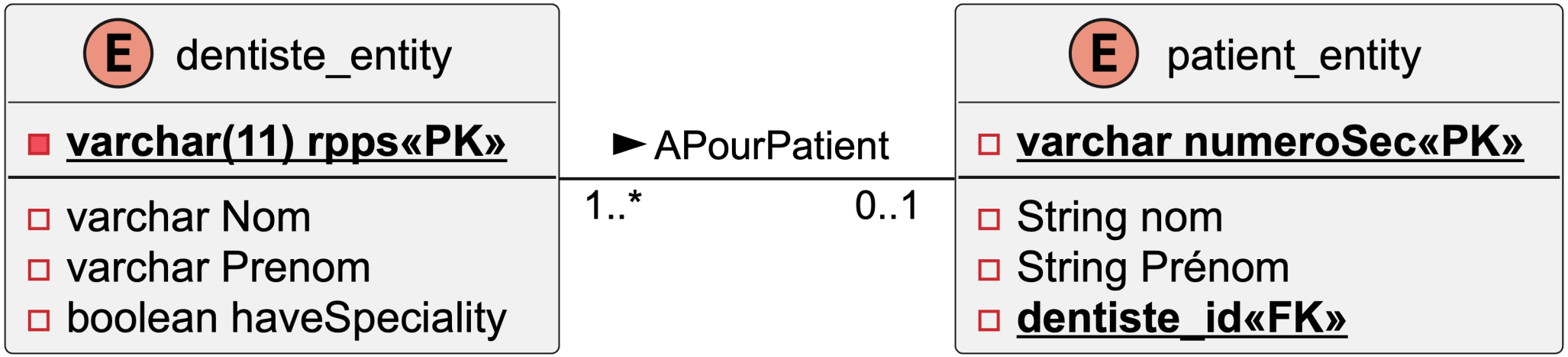


Bidirectionnelle

```java
@Entity
public class PatientEntity{
   @Id
   private String numeroSecu;
   private String nom;
   private String prenom;

   @ManyToOne
   private DentisteEntity dentiste;
}
```

Représentation en base

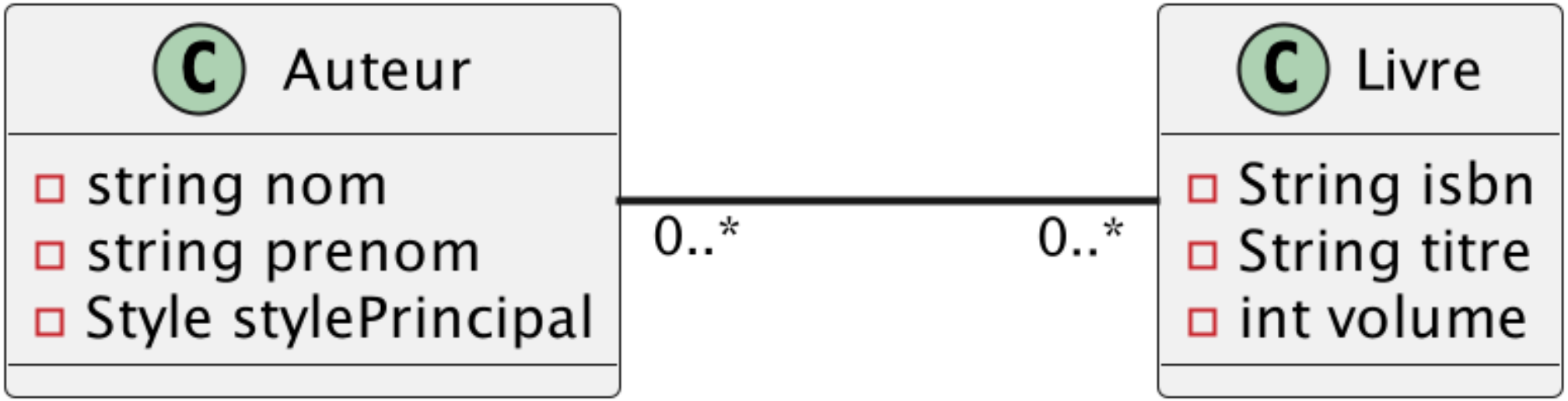


@ManyToMany

Bidirectionnelle

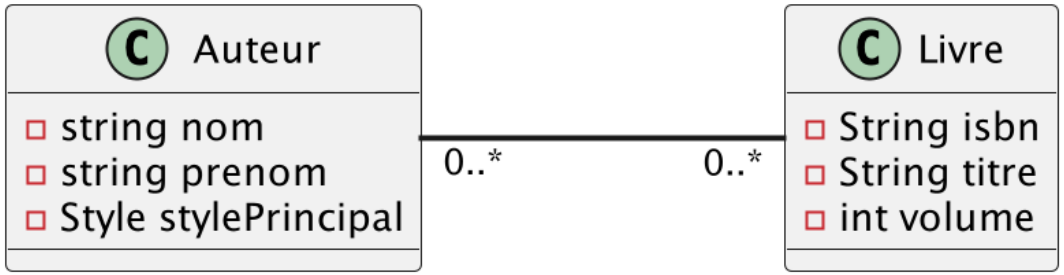
zenika
MIAGE Grenoble RÉSEAU DES MIAGE DE FRANCE
UGA UFR IM²AG Université Grenoble Alpes

@ManyToMany


Auteur
string nom
string prenom
Style stylePrincipal
0..*
0..*
Livre
String isbn
String titre
int volume

@ManyToMany

Bidirectionnelle




```java
@Entity
public class AuteurEntity{
    @Id
    private String nom;
    private String prenom;
    private Style stylePrincipal;

    @ManyToMany
    private Set<LivreEntity> livres;
}
```



Bidirectionnelle

```java
@Entity
public class LivreEntity{
    @Id
    private String isbn;
    private String title;
    private int volume;

    @ManyToMany
    private Set<AuteurEntity> auteurs;
}
```

Représentation en base

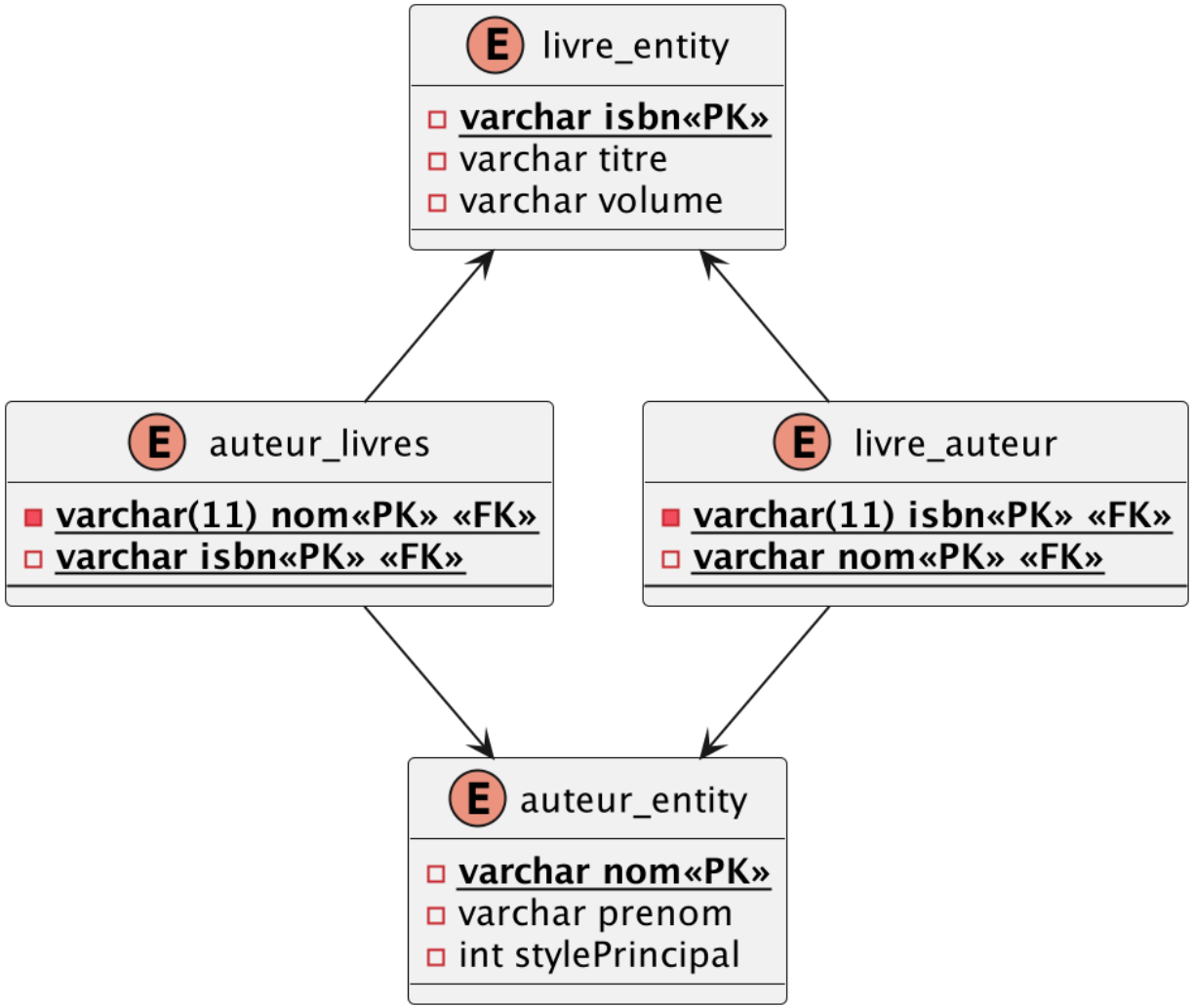


Table d'association

Table d'association

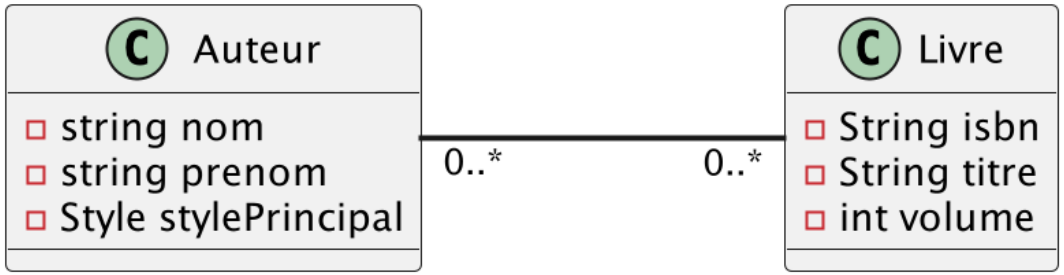
A votre avis, que manque-t-il ?



zenika MIAGE RÉSEAU DES MIAGE DE FRANCE Grenoble UGA UFR IM²AG Université Grenoble Alpes

@ManyToMany

Bidirectionnelle




```java
@Entity
public class AuteurEntity{
    @Id
    private String nom;
    private String prenom;
    private Style stylePrincipal;

    @ManyToMany(mappedBy = "auteurs")
    private Set<LivreEntity> livres;
}
```



Bidirectionnelle

```java
@Entity
public class LivreEntity{
    @Id
    private String isbn;
    private String title;
    private int volume;

    @ManyToMany
    private Set<AuteurEntity> auteurs;
}
```

Représentation en base

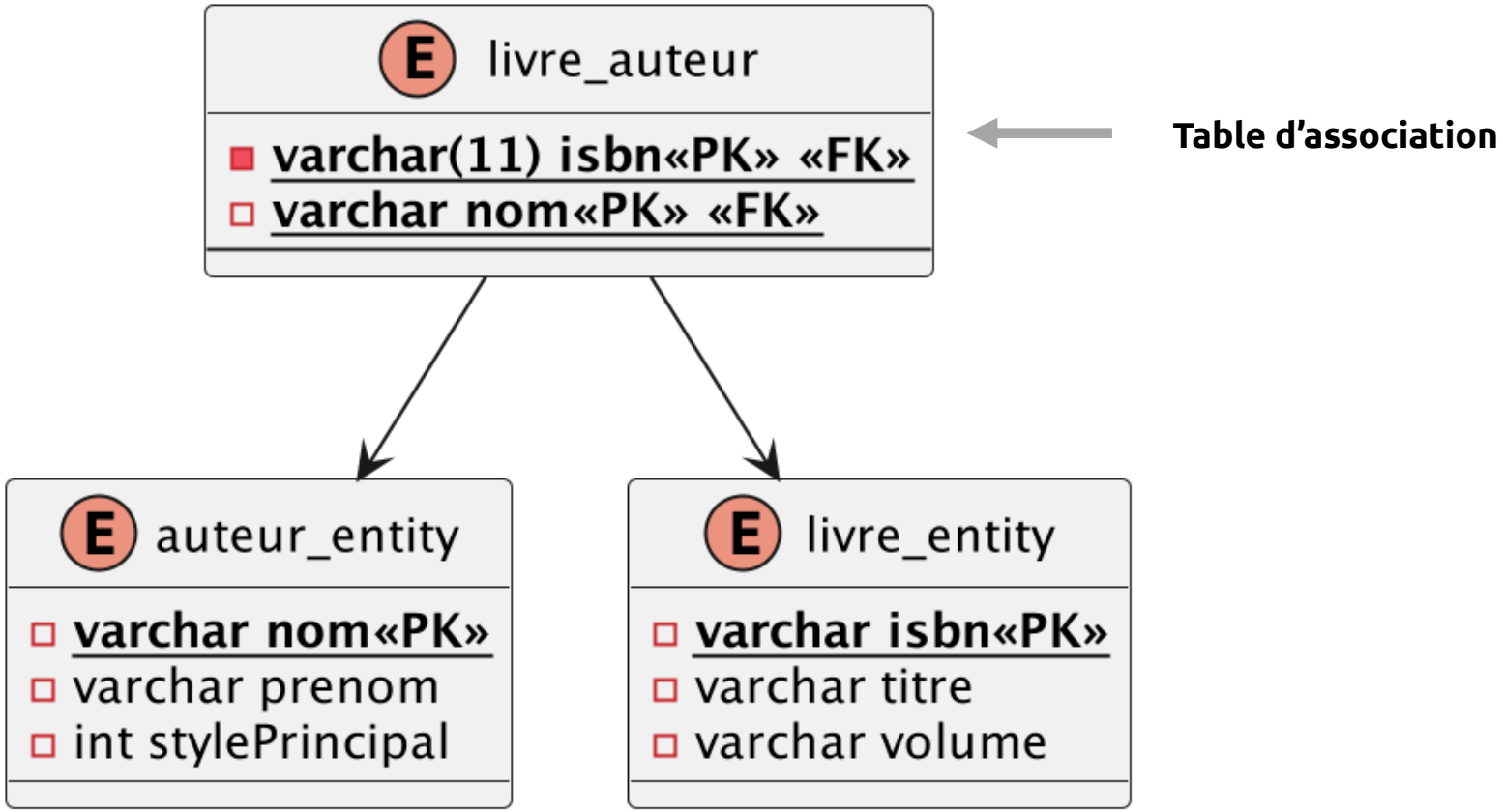
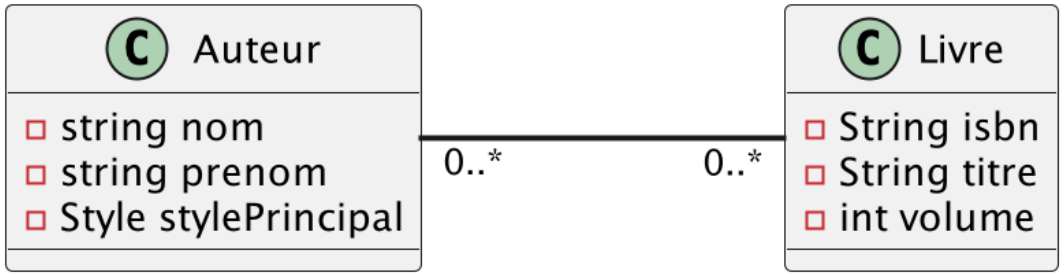


Table d'association

@ManyToMany

Bidirectionnelle




```java
@Entity
public class AuteurEntity{
   @Id
   private String nom;
   private String prenom;
   private Style stylePrincipal;

   @ManyToMany
   private Set<LivreEntity> livres;
}
```



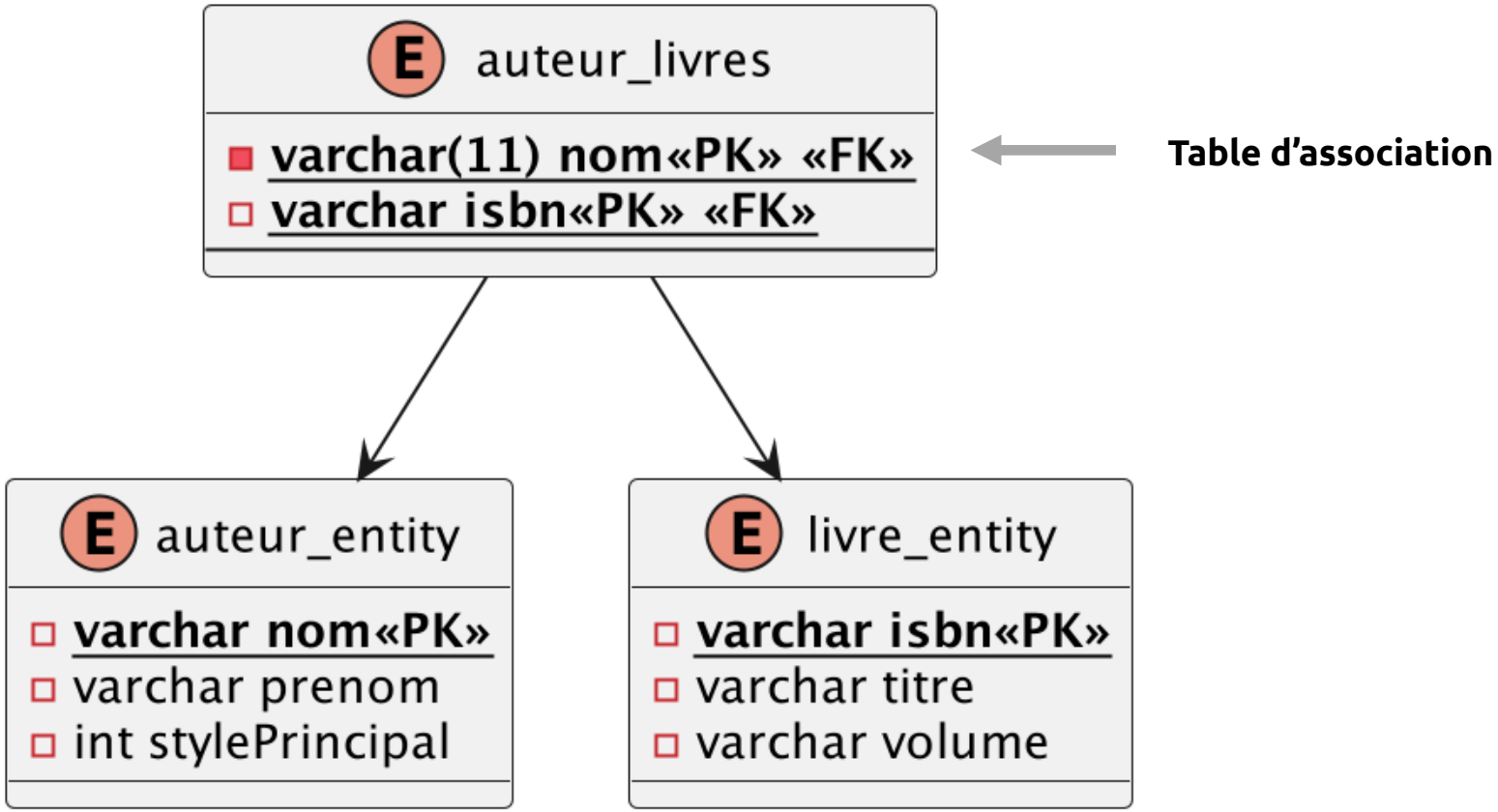
Bidirectionnelle

```java
@Entity
public class LivreEntity{
   @Id
   private String isbn;
   private String title;
   private int volume;

   @ManyToMany(mappedBy = "livres")
   private Set<AuteurEntity> auteurs;

}
```

Représentation en base



On y est presque !



zenika MIAGE RÉSEAU DES MIAGE DE FRANCE Grenoble UGA UFR IM²AG Université Grenoble Alpes

Manipulation des entités

zenika
MIAGE Grenoble RÉSEAU DES MIAGE DE FRANCE
UGA UFR IM²AG Université Grenoble Alpes

Egalité
2 types possibles



Naturelle

- Correspond au **.equals()** en java
- Celle utiliser dans les **Collections** java



En base

- Correspond à l'**id** en base

Fetch



Les relations JPA



EAGER

- Les données sont chargées directement
- Peut-être dangereux niveau code



LAZY

- Les données sont chargées au seul moment où vous les appelez.
- Exemple => **monObjet.GetmaRelation()**
- **Nécessite une transaction**

CASCADE



Les relations JPA



Persist



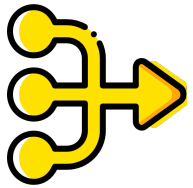
Remove



Detatch



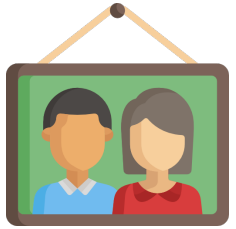
Refresh



Merge







Les parents

Persist

Appliquer l'état du parent à l'enfant



Enfants

Les repositories

JPA Repository



Interface java

JpaRepository<T, ID>



Standard

- save()
- findAll()
- findAllById()
- getOne()
- delete



Création de requête par interpolation du nom de la fonction



Création de requête JPQL, voir même SQL

Implémentation

```java
@Repository
public interface DentisteRepository extends JpaRepository<DentisteEntity,, String>{

    Optional<DentisteEntity> findDentisteByNomAndPrenom(String nom, String prenom);

    @Query("select d from Dentiste d where d.nom = ?1 and d.prenom = ?2")
    DentisteEntity getWithJpqlQuery(String nom, String prenom);

    @Query(value = "select * from public.dentiste_entity d where d.nom = ?1 and d.prenom = ?2",nativeQuery = true)
    DentisteEntity getWithSqlQuery(String nom, String prenom);
}
```

Les fonctions nommés

zenika
MIAGE Grenoble RÉSEAU DES MIAGE DE FRANCE
UGA UFR IM²AG Université Grenoble Alpes

Structures

find[Top|First][N]By Propriete [And|Or Propriete] [MotCle] [OrderBy Champ Asc|Desc]

Collection< AlbumEntity> findAllByArtistEntityAndTitle(ArtistEntity artistEntity, String title);

Collection< AlbumEntity> findAllBy	ArtistEntity	And	(ArtistEntity artistEntity, String title);
Action avec cardinalité	Les conditions en accédant au propriétés	Une liaison pour une autre condition	Les paramètres de la conditions

- And
- Or

Les mots clés

Action avec cardinalité

Objet de retour	Mot clé	Description
Optional<T>	findBy	Vous chercher un élément ou vous n'êtes pas sur qu'il existe
Collection<T>, Page<T>	findAllBy	Vous chercher plusieurs élément, si aucun existe, alors la liste de retour est vide
T	getBy	Vous voulez récupérer un élément où vous êtes sur qu'il existe
Boolean	existsBy	Vous voulez vérifier qu'un élément existe
Integer	countBy	Vous voulez compter le nombre d'élément
Integer	deleteBy	Vous voulez supprimer des éléments. Le retour correspond au nombre de tuple supprimer

⚠ Peux aussi avoir des condition spécial avant le **by**

Spécial	Description	Utilisation
Distinct	Faire un distinct sur la requête	findDistinctBy
Top	Récupérer les premiers tuples	Avec 5 : findTop3
First	Récupérer le premier	findFirst

Les mots clés

Comparaison
Is
Equals
IsNot
Not
IsNull
IsNotNull
Null
NotNull

booléens
True
False

Texte
Like
NotLike
Containing
NotContaining
StartingWith
EndingWith
IgnoreCase
IgnoringCase

Collections
In
NotIn
Contains
NotContains
IsEmpty
IsNotEmpty
Empty
NotEmpty

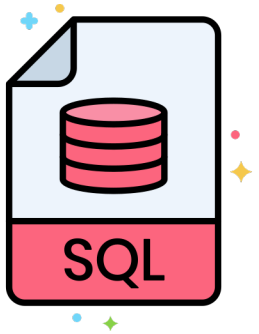
Date / numérique
LessThan
LessThanEqual
GreaterThan
GreaterThanEqual
Before
After
Between

Mise en ordre
OrderBy
Asc
Desc

JPQL

zenika
MIAGE Grenoble RÉSEAU DES MIAGE DE FRANCE
UGA UFR IM²AG Université Grenoble Alpes

JPQL



- Même opérateur
- Même fonctions :
  - Aggregations : COUNT(), SUM()...
  - Arithmétiques : ABS(), SQRT()...
  - Chaîne de caractères: CONCAT(), SUBSTRING()...
- Comparateurs : =, !=, >, <, IN, LIKE, IS NULL ...
- Opérateurs logiques: AND, OR, NOT
  Jointure : JOIN



Utilisation des objets



A utiliser avec parcimonie

Implémentation

```
select b from Book b where b.title = :title
```

```
select d
from Driver d join b.service s
where d.professional=true and s.brand=”Ford”
```

```
select b from Book b where b.title = ?1
```

```
select t.title
from Disk d join d.tracks t where
d.releaseYear=?1
```

```
select new fr.diskworld.SimpleTrack(t.title, t.lengthSec) from Disk
d join d.tracks t where d.releaseYear=?1
```

Bravo ! Vous êtes encore vivants



zenika MIAGE Réseau des MIAGE de France Grenoble UGA UFR IM²AG Université Grenoble Alpes

Quizz final



zenika MIAGE Réseau des MIAGE de France Grenoble UGA UFR IM²AG Université Grenoble Alpes